SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

QUESTIONS: 5 **Total Marks : 50**

Annexure A

Scenario based Questions

Code of Conduct:

I M. Pranay (192525382)_ certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

M. Pranay

SAMPLE QUESTIONS

(QUESTIONS CAN BE CHANGED AS PER YOUR REQUIREMENTS)

Question 1 — CPU Scheduling in a Real-Time Ticket Booking System

A railway ticket booking system experiences heavy load during festival seasons. The system handles three categories of processes:

- P1 – Payment gateway transactions (High priority)
- P2 – Seat availability checks (Medium priority)
- P3 – Ticket confirmation and printing (Low priority)
- Burst times vary from 2 ms to 20 ms.

Recently, customers complained that ticket confirmations are taking too long.

Tasks

1. Identify the most suitable scheduling algorithm (FCFS, SJF, Priority, RR) for this scenario.
2. Justify *why* your chosen method improves CPU utilization and reduces customer wait time.
3. Provide a sample execution order for the above categories.
4. Discuss one drawback of this scheduling method in real-time systems.

(10 Marks) (BL 4)

Question 2 — Deadlock in a Banking Transaction Server

A bank server handles the following operations simultaneously:

- T1: Transfers money between accounts
- T2: Updates passbook entries
- T3: Generates monthly statements

Due to simultaneous locking of Account Database (A), Customer Info Database (B), and Transaction Logs (C), the system enters a state where:

- T1 holds A and requests B
- T2 holds B and requests C
- T3 holds C and requests A

Tasks

1. Draw the resource request situation (text-based RAG).

2. Identify whether a deadlock has occurred and justify your answer.
3. Recommend *one specific technique* (avoidance/prevention/recovery) and explain how it fixes the issue.
4. Suggest a redesign for the locking order to avoid future deadlocks.

(10 Marks) (BL 4)

Question 3 — Mobile OS App Switching Lag

A mobile operating system is designed to support smooth app switching.

But users report that:

- Opening WhatsApp delays the background music player.
- Switching between social media apps feels laggy.
- CPU shows high idle times despite multiple running apps.

The OS currently uses Round Robin (quantum = 20 ms) with 20 background apps active.

Tasks

1. Analyze why the CPU remains idle despite many active processes.
2. Determine whether the time quantum is helping or hurting performance.
3. Recommend a better scheduling method or quantum value, with justification.
4. Explain how the change will reduce switching lag and improve system throughput.

(10 Marks) (BL 4)

Question 4 — Deadlock Risk in an Operating System Update

During a Windows OS update, several modules run in parallel:

- M1: Copy update files
- M2: Install drivers
- M3: Update registry
- M4: Validate system integrity

Resources involved:

- R1 = Disk I/O
- R2 = Kernel lock
- R3 = Configuration lock

Recently logs show:

- M1 holds R1, waiting for R2
- M2 holds R2, waiting for R3
- M3 holds R3, waiting for R1

Tasks

1. Identify whether this sequence can lead to a deadlock.
2. If yes, explain the exact circular wait condition.
3. Propose *Banker's algorithm style* resource allocation to avoid risk.
4. Suggest how OS designers can prioritize modules to improve CPU utilization during update.

(10 Marks) (BL 4)

Question 5 — Cloud Server CPU Utilization Analysis

A cloud hosting provider notices:

- 80% of the time, VM processes wait for network or disk
- Only 20% of the time is spent on CPU
- Each server runs 7 VMs

Using the formula:

where p = I/O wait probability

Tasks

1. Calculate CPU utilization.
2. Interpret whether the server is underutilized or optimally loaded.
3. Suggest how increasing or decreasing number of VMs affects throughput.
4. Propose two OS-level strategies for improving CPU utilization in this cloud environment.

(10 Marks) (BL 4)

RUBRICS FOR CALCULATION- Scenario based assessment

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding ; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

TOTAL MARKS AWARDED **FACULTY SIGNATURE**

ANSWERS

1. CPU Scheduling in a Real-Time Ticket Booking System

1. Suitable scheduling algorithm

Priority Scheduling is suitable because the three process categories have different importance levels: P1 (payment gateway) has high priority, P2 (seat availability) has medium priority, and P3 (ticket confirmation and printing) has low priority. A preemptive priority scheduler can run a newly ready high-priority payment task without waiting for a lower-priority task to finish. Aging should be added so that P3 does not wait indefinitely.

2. How it improves CPU utilization and waiting time

Important payment transactions receive CPU time promptly, helping the system process time-sensitive work.

When a process blocks for I/O, the scheduler can run another ready process, keeping the CPU busy.

Giving seat checks medium priority helps availability information reach customers before final confirmation.

Priority scheduling reduces waiting for critical tasks, although low-priority tasks may wait longer.

3. Sample execution order

Assuming all three processes are ready at the same time and remain ready, a sample order is P1 → P2 → P3. If a high-priority P1 process becomes ready while P2 or P3 is running, preemptive priority scheduling can pause the lower-priority process and run P1.

4. One drawback

Starvation is the main drawback: continuous arrivals of high-priority payment tasks may prevent low-priority ticket confirmation and printing from running. Aging, which gradually raises the priority of waiting tasks, helps reduce this risk.

2. Deadlock in a Banking Transaction Server

1. Text-based Resource Allocation Graph (RAG)

Allocation edges (resource → transaction): A → T1, B → T2, C → T3
Request edges (transaction → resource): T1 → B, T2 → C, T3 → A
Cycle: T1 → B → T2 → C → T3 → A → T1

2. Has a deadlock occurred?

Yes. T1 holds resource A and waits for B, which is held by T2. T2 holds B and waits for C, which is held by T3. T3 holds C and waits for A, which is held by T1. Each transaction is waiting for a resource held by another transaction in the same cycle. Assuming each resource has a single instance, the cycle confirms deadlock.

3. Recommended technique: Deadlock recovery

The server can select one transaction as a victim, roll it back, and release its locks. For example, rolling back T3 releases C, allowing T2 to proceed; after T2 releases B, T1 can continue. The victim should be chosen using factors such as work already completed, rollback cost, and transaction priority. The system should log the rollback and retry the transaction safely.

4. Redesign the locking order

Define one global lock order, for example A → B → C, and require every transaction to acquire locks only in that order. A transaction needing multiple resources must request A before B and B before C, rather than taking them in different orders. Consistent ordering prevents the circular-wait condition.

3. Mobile OS App Switching Lag

1. Why the CPU may remain idle

Having 20 active background apps does not mean that all are ready to execute. Many may be waiting for network, storage, timers, or other I/O events. If no process is in the ready queue, the CPU has nothing to execute and may remain idle. Delays can also arise from excessive context switching, memory pressure, or slow I/O.

2. Effect of a 20 ms time quantum

A 20 ms quantum can be relatively long for an interactive mobile system: an app may hold the CPU long enough for other interactive tasks to feel delayed. However, making the quantum too short increases context-switch overhead. Therefore, the quantum should be tuned to workload and device measurements rather than reduced without limit.

3. Recommended scheduling method

Use a Multilevel Feedback Queue (MLFQ) or another priority-aware preemptive scheduler. Give interactive and I/O-bound tasks, such as audio playback and foreground app interactions, prompt service, while background tasks receive CPU time at lower priority. As a starting point, test a shorter interactive quantum such as 5–10 ms and measure responsiveness and context-switch overhead.

4. Expected improvements

Foreground interactions receive CPU time sooner, reducing perceived app-switching lag.

The music player can be scheduled promptly when it becomes ready, improving playback continuity.

Short, responsive CPU bursts improve perceived responsiveness, while longer background quanta can limit overhead.

The scheduler should also monitor I/O delays and memory pressure, because scheduling alone cannot fix a system with no ready tasks.

4. Deadlock Risk in an Operating System Update

1. Can the sequence lead to deadlock?

Yes. The modules form a circular wait: M1 holds R1 and requests R2; M2 holds R2 and requests R3; M3 holds R3 and requests R1. If each resource is a single-instance, non-preemptable resource, all three modules can wait forever.

2. Exact circular-wait condition

R1 → M1 → R2 → M2 → R3 → M3 → R1. The resource-allocation edges show R1 allocated to M1, R2 to M2, and R3 to M3; the request edges close the cycle. The cycle means each module waits for a resource held by the next module.

3. Banker's algorithm-style allocation

Before granting a resource request, record each module's maximum resource requirement, current allocation, and remaining need.

Temporarily consider the requested allocation and calculate the available resources and each module's remaining need.

Run a safety check: look for a sequence in which each module's remaining need can be met by currently available resources plus resources released by previously completed modules.

Grant the request only if a safe completion sequence exists; otherwise, make the module wait. This avoids entering an unsafe state that could lead to deadlock.

For example, if granting M1 R2 would leave no possible completion sequence because M2 and M3 are still waiting, the request should be delayed until a safe allocation is possible. Banker's algorithm requires known maximum claims and suitable resource accounting.

4. Prioritizing modules to improve CPU utilization

Give appropriate priority to modules on the critical update path, but do not allow a high-priority module to hold locks indefinitely. Use bounded lock waits, release resources as soon as possible, and schedule other ready work while a module waits for disk I/O. A safe resource-allocation policy and careful I/O scheduling improve progress and reduce wasted CPU time.

5. Cloud Server CPU Utilization Analysis

1. Calculate CPU utilization

For n independent processes/VMs, where each process waits for I/O with probability p, the standard multiprogramming model is: CPU Utilization = $1 - p^n$.

Given: p = 0.80 and n = 7 VMs

CPU Utilization = $1 - (0.80)^7 = 1 - 0.2097152 = 0.7902848$

Estimated CPU utilization = 79.03%

This is the estimate under the formula's assumptions. It is not the same as simply saying that each VM uses the CPU 20% of the time; multiple VMs can keep the CPU busy while others wait for I/O.

2. Is the server underutilized or optimally loaded?

An estimated utilization of about 79% suggests the server is reasonably well utilized, with some CPU idle time remaining. Whether it is optimal depends on response-time targets, memory use, network and disk bottlenecks, and workload variation. CPU utilization alone is not enough to decide.

3. Effect of increasing or decreasing the number of VMs

Increasing the number of VMs can improve throughput when existing VMs often wait for I/O, because another ready VM can use the CPU.

Too many VMs can cause memory pressure, scheduling overhead, and contention for disk or network resources, reducing performance.

Decreasing the number of VMs can reduce contention, but may leave the CPU idle more often if the remaining VMs are waiting for I/O.

4. Two OS-level strategies

Improve I/O handling: use asynchronous I/O, efficient disk queues, and appropriate network buffering to reduce avoidable waiting and improve overlap between I/O and CPU work.

Tune scheduling and multiprogramming: use fair, workload-aware CPU scheduling and adjust VM concurrency based on CPU, memory, and I/O monitoring so that ready tasks can run without excessive contention.

Conclusion

Effective CPU scheduling improves responsiveness and throughput when ready processes are available. Deadlocks can be handled through recovery and prevented by consistent resource ordering or avoided through safe-state checks. In cloud systems, CPU utilization should be evaluated alongside I/O, memory, and response-time measurements.